

HACKMTY 2026

The Forensic Auditor

Infosys Challenge Track

DOCUMENTO DE ARQUITECTURA

Fase 3 de 4 · Investigación → Planeación → Arquitectura → Desarrollo

Este documento es el contrato que se congela en H+4

Qué es este documento

Arquitectura es decidir tres cosas. Nada más:

1. Qué tablas tiene la base de datos y qué columnas.
2. Qué puede hacer el agente, es decir qué preguntas le puede hacer a esos datos.
3. En qué formato entrega el resultado al frontend.

Esas tres cosas son el contrato. Una vez acordadas, cada persona construye su parte sin depender del código de las demás. Por eso se congelan en H+4 y no se vuelven a tocar.

Todo lo demás —cómo se ve el grafo, qué librería se usa, cómo se escribe un prompt— se decide sobre la marcha y no rompe a nadie.

El stack

Backend	Python (FastAPI)
Almacén	PostgreSQL local -> Tiger Data
Modelo	Ollama local (Mac M4) + Gemini
Frontend	Vite + React
Despliegue	Vultr, en paralelo

IMPLICACIÓN DE DISEÑO — local manda, la nube es paralela

El demo corre en local, en la Mac M4. Tiger Data y Vultr son despliegues paralelos para cumplir las categorías de patrocinador y para mostrar en la arquitectura.

Razón: la red del evento va a estar saturada con cientos de personas. Quince consultas cruzando internet durante el demo es un riesgo que no hace falta correr.

Requisito: el sistema debe correr en ambos lados sin cambiar código. Solo variables de entorno para la URL de la base y la de Ollama.

IMPLICACIÓN DE DISEÑO — el equipo es mixto

Dos Mac y dos Windows. Cuidado con rutas absolutas y con dependencias que se comporten distinto entre sistemas.

La Mac M4 es la máquina del demo, así que el backend debe correr ahí desde temprano. No descubrir a las dos de la mañana que algo no compila en ARM.

1 — Vista general

```
[archivo del jurado]
|
CARGA -> PostgreSQL
|
DETECTORES DETERMINISTAS (SQL, sin modelo)
|   ranking de candidatos
LOOP DEL AGENTE           (modelo + herramientas)
|   hipótesis -> consulta -> evidencia
GATE DE VEREDICTO         (reglas, sin modelo)
|
CASE FILE -> JSON -> React
```

Dos decisiones están incorporadas en ese diagrama y conviene explicitarlas.

- **Los detectores van antes del modelo.** Son consultas baratas que ordenan a los proveedores por señales obvias, para que el agente solo gaste llamadas en lo que vale la pena. Con sesenta segundos de presupuesto, no se puede investigar a los cuarenta.
- **El gate va después, en código.** El veredicto no lo decide el modelo de lenguaje. Lo decide una función con reglas explícitas. El modelo decide a quién investigar y redacta el razonamiento; no etiqueta.

IMPLICACIÓN DE DISEÑO — qué hace y qué no hace el modelo

SÍ hace:

- decide a quién investigar y en qué orden
- formula la hipótesis
- decide cuándo abandonar un lead
- redacta el razonamiento y el case file

NO hace:

- calcular montos o porcentajes
- asignar el veredicto
- leer el ground truth

Esto no es purismo: es lo que hace que el resultado sea defendible cuando el jurado pregunte de dónde salió una cifra.

2 — El contrato de datos

Nueve tablas, agrupadas por origen. Esto es lo que se congela en H+4.

Generadas por nosotros

```
proveedor
  rfc PK, nombre, giro, fecha_alta,
  es_cliente_tambien BOOL

cfdi
  uuid PK, fecha, fecha_timbrado,
  rfc_emisor FK, rfc_receptor,
  tipo_comprobante, forma_pago, metodo_pago,
  uso_cfdi, subtotal, total, moneda

cfdi_concepto
  id PK, uuid FK, clave_prod_serv,
  cantidad, descripcion, valor_unitario, importe

soporte_documental
  id PK, uuid FK, tipo, fecha,
  referencia, monto_amparado
  -- tipo: contrato | orden_compra | entrega

ledger
  id PK, uuid FK, periodo, cuenta_contable,
  monto_deducido, iva_acreditado
```

Nota sobre soporte_documental: no generamos documentos reales. Generamos el registro de que existen o no, con metadatos que el agente pueda cruzar. El campo monto_amparado es importante: un contrato por 800 mil contra facturas por 2.4 millones es materialidad parcial, que es la firma del sobreprecio con retorno.

El puente factura ↔ banco

```
cfdi_pago          -- el complemento REP
  id PK, uuid_factura FK, fecha_pago,
  monto, forma_pago, tran_id FK

cuenta
  acct_id PK, rfc, banco,
  titular_tipo,      -- fisica | moral
  vinculo_con        -- rfc de la empresa
                    -- investigada, si aplica
  tipo_vinculo       -- socio | directivo |
                    -- filial | ninguno
```

Un RFC puede tener varias cuentas. Eso es realista y además es un vector de fraude: dispersar pagos entre cuentas del mismo titular para no llamar la atención.

IMPLICACIÓN DE DISEÑO — los campos de vínculo son nuevos y son críticos

titular_tipo y vinculo_con no estaban en el diseño inicial. Se agregaron al resolver la pregunta de cuándo un retorno de dinero es fraude y cuándo no.

Sin ellos no se puede distinguir «le pagué a otra empresa» de «el dinero llegó a la cuenta personal del socio». Y esa distinción es la que sostiene la acusación.

De AMLSim

```
transaccion
  tran_id PK, orig_acct FK, bene_acct FK,
  monto, fecha
```

Se normaliza en la carga. El dataset pregenerado y la salida actual de AMLSim usan nombres de columna distintos; ambos se convierten a este esquema.

Del SAT

```
efos_evento
  id PK, rfc, etapa,
  fecha_dof, fecha_portal, oficio
  -- etapa: presunto | definitivo |
  -- desvirtuado | sentencia_favorable
```

Es la tabla larga, pivotada desde el CSV del SAT. Una fila por evento, no por contribuyente. Así la pregunta «¿qué estatus tenía este RFC el día D?» es una consulta ordinaria en vez de un CASE de quince ramas.

Fuera del alcance del agente

```
gt_escenario
  id, tipo, rfc_proveedor, uuids[], acct_ids[]
  -- tipo: legitimo | desordenado |
  -- simulador | reciproco
```

IMPLICACIÓN DE DISEÑO — el ground truth vive aparte

Esta tabla va en otro esquema de base de datos o con permisos distintos. El usuario con el que se conecta el agente no la puede leer.

Si el agente puede leer la respuesta correcta, la métrica no vale nada. Esto no es una precaución teórica: es fácil que se cuele en un SELECT * durante el desarrollo.

Los cuatro tipos de proveedor

El data estate debe sembrar los cuatro obligatoriamente. Cada uno existe para que el agente enfrente una decisión distinta.

Tipo	Cómo se construye	Veredicto esperado
Legítimo	Contrato, orden de compra y evidencia de entrega completos. Pago único sin retorno.	DEFENDIBLE
Desordenado	Sin contrato, pero con evidencia de entrega. Pago único, sin retorno. Puede estar en estatus definitivo.	CORREGIR
Simulador	Sin contrato ni entrega. El pago se dispersa y retorna a una cuenta vinculada, sin factura recíproca.	ACUSACIÓN
Recíproco	Proveedor que también es cliente. Hay dinero de vuelta, pero con facturas propias en sentido contrario, a cuenta de persona moral.	DEFENDIBLE

El desordenado y el recíproco son los dos casos que hacen interesante el proyecto. Los dos generan señales que un detector ingenuo interpretaría como fraude, y los dos deben salir limpios.

Índices

Con sesenta segundos de presupuesto esto no es opcional.

```
CREATE INDEX ON cfdi(rfc_emisor, fecha);
CREATE INDEX ON cfdi(rfc_receptor, fecha);
CREATE INDEX ON transaccion(orig_acct, fecha);
CREATE INDEX ON transaccion(bene_acct, fecha);
CREATE INDEX ON efos_evento(rfc, etapa);
CREATE INDEX ON soporte_documental(uuid);
CREATE INDEX ON cfdi_pago(uuid_factura);
```

3 — Las herramientas del agente

Siete funciones. Ni una más: cada herramienta adicional es otra decisión que el modelo puede tomar mal, y otra llamada que gasta presupuesto de latencia.

```
candidatos_priorizados() -> list
    detectores deterministas, sin modelo.
    devuelve proveedores ordenados por señal.

estatus_efos(rfc, fecha_factura) -> dict
    estatus vigente en esa fecha, si aplica
    retroactividad, y el número de oficio.

facturas_de(rfc_proveedor) -> list
    facturas, montos, conceptos, claves.

materialidad(uuid) -> dict
    contrato / entrega / pago: sí o no,
    con sus referencias y montos amparados.

rastrear_dinero(uuid, max_saltos=4) -> dict
    WITH RECURSIVE sobre transaccion.
    devuelve ruta, monto_retornado,
    pct_retorno, cuenta_destino,
    titular_tipo y vinculo.

hay_reciprocidad(rfc_a, rfc_b, periodo) -> dict
    ¿existen CFDI en sentido contrario que
    justifiquen el retorno? montos y folios.

deduccion(uuid) -> dict
    si se dedujo, en qué periodo,
    monto en pesos e IVA acreditado.
```

IMPLICACIÓN DE DISEÑO — las herramientas devuelven datos, nunca juicios

materialidad() responde «contrato: no». No responde «sospechoso».

rastrear_dinero() responde «retorno 92.9%, cuenta 4471, titular persona física, vínculo socio». No responde «esto es fraude».

La interpretación ocurre en el gate, que es código con reglas visibles. Si el juicio vive dentro de la herramienta, deja de ser auditable.

IMPLICACIÓN DE DISEÑO — hay_reciprocidad es la herramienta que evita el falso positivo

Un retorno de dinero puede ser perfectamente legítimo: compraventa recíproca, empresas del mismo grupo, devoluciones, reembolsos.

Lo que distingue el retorno legítimo del simulado es que el legítimo tiene su propia factura. Si te pago 2.4 millones y me pagas 2.2, hay dos CFDI, uno de cada lado. En la simulación no puede haberlos, porque no hay operación que facturar.

Sin esta herramienta, el agente acusaría a todo proveedor que también sea cliente. Con ella, ese caso se descarta con una razón que un auditor reconoce.

La frontera de responsabilidad

La carpeta tools/ es el límite entre dos personas. Quien escribe las consultas entrega estas siete funciones; quien escribe el agente las consume. Nadie más las toca.

Consecuencia práctica: quien construye el agente puede empezar con versiones falsas de estas siete funciones —que devuelvan datos escritos a mano— y avanzar sin esperar a que la base esté lista.

4 — El loop y el gate

El loop de investigación

1. candidatos_priorizados()
-> proveedores ordenados por señales baratas
2. tomar los primeros N (N ~ 8, ajustable según latencia medida)
3. por cada candidato:
 - a. estatus_efos(rfc, fecha)
si desvirtuado o sentencia favorable:
DESCARTA y registra el motivo
 - b. facturas_de(rfc)
 - c. materialidad(uuid) por factura relevante
 - d. si falta materialidad:
rastrear_dinero(uuid)
 - e. si hay retorno:
hay_reciprocidad(empresa, titular_destino)
 - f. si el hallazgo se sostiene:
deduccion(uuid)
4. gate_veredicto(evidencia) -> etiqueta
5. ensamblar case file

El paso 3a es el que mata candidatos barato, antes de gastar llamadas. Y cada descarte se registra con su motivo, que es lo que alimenta la sección de leads no perseguidos del case file.

El gate de veredicto

Esto vive en código, no en el prompt.

```
def veredicto(ev):  
  
    # el SAT ya lo exoneró  
    if ev.estatus in ("desvirtuado",  
                     "sentencia_favorable"):  
        return "LIMPIO"  
  
    # materialidad completa  
    if ev.contrato and ev.entrega and ev.pago_limpio:  
        return "DEFENDIBLE"  
  
    # retorno con sustento: no es hallazgo  
    if ev.retorno and ev.reciprocidad_documentada:  
        return "DEFENDIBLE"  
  
    # retorno sin sustento a cuenta vinculada  
    if (ev.retorno  
        and not ev.reciprocidad_documentada  
        and ev.cuenta_destino_vinculada):
```

```
return "ACUSACION"
```

```
# todo lo demás: conservador  
return "CORREGIR"
```

IMPLICACIÓN DE DISEÑO — no hay umbral de porcentaje

El diseño inicial contemplaba acusar cuando el retorno superara cierto porcentaje. Se descartó.

Cualquier número es arbitrario y no se puede defender: si el jurado pregunta por qué 60 y no 55, no hay respuesta. Es la misma objeción que aplica a un score de probabilidad.

El criterio final no usa umbrales. Usa tres condiciones binarias: hay retorno, no hay factura recíproca que lo justifique, y la cuenta destino está vinculada al pagador. Eso se defiende ante cualquier auditor.

El porcentaje sigue existiendo, pero como dato del reporte, no como criterio de decisión.

IMPLICACIÓN DE DISEÑO — el default es conservador

Cuando la evidencia no alcanza para ninguna rama clara, el veredicto es CORREGIR, no ACUSACIÓN.

Es la traducción directa del criterio de juicio del track: no acusar a quien no se puede respaldar. Un falso positivo le cuesta a la empresa cortar a un proveedor bueno o señalar a un empleado inocente.

Presupuesto de latencia

El demo da entre sesenta y noventa segundos de cómputo, cubiertos por la narración. De ahí sale el presupuesto.

Medir en la primera hora:

1. correr una llamada al modelo
2. cronometrarla
3. multiplicar por 15

Si no cabe, bajar de modelo:

qwen2.5:7b o llama3.1:8b (con GPU)
qwen2.5:3b (si va lento)

La Mac M4 con memoria unificada corre un modelo de 7 a 8 mil millones de parámetros sin problema. El riesgo real no es la capacidad sino la velocidad acumulada de quince llamadas.

El caché por hash del prompt es lo que salva los ensayos: la segunda corrida del mismo archivo es instantánea.

5 — El contrato de salida

Lo que el backend entrega y el frontend consume. Acordar esto en H+4 permite que quien hace la interfaz trabaje con un JSON escrito a mano desde el primer minuto, sin esperar a que el agente funcione.

```
{
  "resumen": {
    "investigados": 40,
    "acusacion": 8, "corregir": 5,
    "defendible": 27, "limpio": 12,
    "monto_total": 14200000.00,
    "duracion_seg": 47
  },
  "destacados": {
    "acusacion": "CDN190312AB1",
    "corregir": "TRE180422XY3"
  },
  "proveedores": [{
    "rfc": "CDN190312AB1",
    "nombre": "CONSULTORES DEL NORTE SA",
    "veredicto": "ACUSACION",
    "monto": 2400000.00,
    "evidencia": [
      {"check": "contrato", "ok": false,
        "detalle": "sin contrato ni orden de compra"},
      {"check": "entrega", "ok": false,
        "detalle": "sin evidencia de entrega"},
      {"check": "pago", "ok": false,
        "detalle": "retorno 92.9%, 3 saltos, 12 días,
          cuenta 4471, socio mayoritario,
          sin factura recíproca"},
      {"check": "efos", "ok": true,
        "detalle": "definitivo DOF 2025-08-20,
          oficio 500-05-2025-XXXXX"}
    ]
  }],
  "grafo": {
    "nodos": [{ "id": "...", "etiqueta": "...",
      "tipo": "proveedor",
      "veredicto": "ACUSACION", "paso": 3 }],
    "aristas": [{ "origen": "...", "destino": "...",
      "monto": 800000,
      "fecha": "2024-04-02", "paso": 4 } ]
  },
  "descartados": [
    { "rfc": "MDG...",
      "motivo": "desvirtuado 2024-11-03" }
  ],
}
```

```
"razonamiento": [  
  {"paso":1,"accion":"...", "hallazgo":"..."}  
]  
}
```

IMPLICACIÓN DE DISEÑO — destacados abre las dos tarjetas solas

El campo destacados es lo que evita hacer clic en vivo durante los tres minutos. El backend elige los dos casos que mejor ilustran el contraste y el frontend los despliega al terminar.

La regla de selección: la acusación con mayor porcentaje de retorno, y el caso de corregir con más evidencia de entrega. No se eligen a mano, porque el archivo del jurado puede cambiar cuáles son los interesantes.

IMPLICACIÓN DE DISEÑO — el campo paso permite la animación sin streaming

Cada nodo y cada arista traen el número de paso en que el agente los descubrió. El frontend recibe el grafo completo y va mostrando los elementos con un retardo.

Es mucho más simple que hacer streaming desde el backend y se ve exactamente igual. Y si aplica el corte de H+14, se quita el retardo y aparece todo de golpe: misma estructura, menos código.

La secuencia del grafo

- | | |
|--------|--|
| Paso 1 | La empresa al centro, proveedores alrededor. Todos grises. |
| Paso 2 | Algunos se atenúan: los descartados. |
| Paso 3 | Los que quedan se marcan con color. |
| Paso 4 | Del sospechoso salen flechas nuevas. Aparecen cuentas intermedias. |
| Paso 5 | Las flechas convergen. |
| Paso 6 | Se revela el nodo destino y el porcentaje de retorno. |

El paso 2 es el diferenciador visual: es el agente descartando. Nadie más va a mostrar lo que no persiguió, y mostrarlo es literalmente un criterio del track.

El paso 6 es el clímax. La revelación del titular de la cuenta se guarda para el final; si desde el paso 4 ya dice «cuenta del socio», se pierde el momento.

Restricciones de la visualización

- Posiciones fijas. Nada de auto-layout que reacomode los nodos al aparecer uno nuevo.
- Sin zoom ni desplazamiento durante el demo. Una vista, completa y legible.
- Máximo 12 a 15 nodos visibles. Con cuarenta no se ve nada.
- Sin animaciones de partículas. Flechas que aparecen, y ya.

6 — Estructura del proyecto

```
backend/  
  ingest/      carga 69-B, AMLSim,  
                archivo del jurado  
  generate/    data estate + inyector  
                de escenarios  
  tools/       las 7 herramientas (SQL puro)  
  agent/       loop, prompts, gate  
  api/         FastAPI: POST /investigar  
  eval/        harness de métrica  
  
frontend/  
  src/         Vite + React
```

tools/ es la frontera entre dos personas: quien escribe las consultas entrega, quien escribe el agente consume. Nadie más entra ahí.

Variables de entorno

Para que el mismo código corra en local y en la nube sin modificarse.

```
DATABASE_URL      postgres local | Tiger Data  
MODEL_BACKEND     ollama | gemini  
OLLAMA_URL        localhost | instancia Vultr  
OLLAMA_MODEL      qwen2.5:7b | qwen2.5:3b  
CACHE_DIR         ruta del caché de prompts
```

Migración a Tiger Data

Tiger Data es PostgreSQL, así que migrar es cambiar la cadena de conexión y correr el script de creación de tablas allá. Cuesta alrededor de media hora, con una condición: no usar funciones específicas de Tiger al principio.

Tablas normales y SQL estándar durante el desarrollo. Al migrar, la tabla de transacciones se convierte en hypertable y se agregan las continuous aggregates. Eso es aditivo, no un rediseño.

Si a H+20 el equipo va tarde, se queda en local y se pierde esa categoría. Nada más.

IMPLICACIÓN DE DISEÑO — decidir en el ensayo contra cuál se presenta

Si Tiger Data está migrado, medir ambas opciones durante el ensayo. Si local es más rápido, se presenta contra local y Tiger se muestra como parte de la arquitectura.

El criterio es el tiempo de la corrida completa, no la elegancia del despliegue.

7 — Lo primero que hay que hacer

Orden concreto para las primeras horas.

Cuándo	Qué	Quién
Primeros 15 min	Descomprimir el dataset pregenerado de AMLSim. Con eso ya hay datos de flujo etiquetados y nadie se bloquea.	Ingesta
Primera hora	Medir la latencia del modelo en la Mac: una llamada, cronómetro, por quince. Decide qué modelo se usa.	Agente
Primera hora	Descargar el CSV del 69-B, imprimir las primeras 20 filas crudas, verificar encoding y alineación de columnas antes de escribir el parser.	Ingesta
H+0 a H+4	Acordar y congelar este documento: las nueve tablas, las siete herramientas y el JSON de salida.	Todos
Desde H+4	Quien hace agente e interfaz trabaja con datos escritos a mano. No espera a que la base esté lista.	Agente, Interfaz

Recordatorio

Tres veredictos accionables,
probados con el flujo del dinero,
medidos con ground truth.

Si una tarea no sirve a esa frase, se corta.